Backblaze Drive Stats Data Platform: Design and Technical Reference

Author: Pranav Aditya · Plan: 29 Sep 2026 · **Final, as built: 13 Oct 2026**

Repository: https://github.com/PRANAV-ADITYA/backblaze-drive-stats-platform

This document describes the platform exactly as it was built and measured. Where the build differs from the original plan, the reason is given in §17.

---

1. Summary

A batch data platform on AWS. It ingests Backblaze's Drive Stats ZIP files (one CSV per day of every operational drive) and turns them into three things:

- a tested Apache Iceberg lake;
- drive-reliability marts, reconciled against every published Backblaze figure;
- point-in-time-correct ML tables.

The pipeline runs daily with no manual steps. Everything is defined in Terraform and changed through pull requests and CI.

Headline	Value
History	13 years: 2013-04-10 → 2026-06-30, 4,830 daily files, **744,290,059 rows**, 12 header layouts
Correctness	All **78** published Backblaze AFR figures recomputed: **74 within tolerance**, 4 gaps documented
Idempotency	Full pipeline rerun gives an identical gold fingerprint
Performance	Silver load for one quarter 1,944 s → 502 s; one-drive lookup 4,043 MB → 115 MB scanned; table 30% smaller
ML	121.8M-row feature table with a 30-day failure label (null at the data horizon), 4 leakage tests
Operations	Email alerts → runbooks, dashboard, monthly maintenance

Scale framing: this is medium data (under 1 B rows, ~17 GB of compressed silver), not multi-TB. It is still big enough that layout and compute choices measurably matter, and every such choice here is backed by a measurement.

2. Source dataset

Aspect	Detail
Grain	One row per operational drive per day
Delivery	Yearly ZIPs for 2013–2015, quarterly ZIPs from 2016; one CSV per day inside
Core columns	`date`, `serial_number`, `model`, `capacity_bytes`, `failure`, plus `smart_N_normalized` / `smart_N_raw` pairs
Added Q2 2023	`vault_id`, `pod_id`, `is_legacy_format`

Aspect	Detail
Added Q3 2023	`datacenter`, `cluster_id`, `pod_slot_num`
License	Free to use with attribution; may not be sold

Defects found by profiling (`docs/profiling-report.md`):

- The 2013 collection gap (2013-08-20 → 2013-10-14).
- Excel-saved files (2018-02-25, 2019-06-17).
- Capacity glitches (Q2 2019).
- Empty daily files (2014-11-02, 2015-11-01, 2017-01-30).
- **2022-12-10 is a copy of 2022-12-11** (§9).

Why not query Backblaze's own Iceberg copy? The ZIPs are treated as a vendor file feed and the system of record; this platform owns the lake, the contract, the quality gates and the business logic.

3. Problem statement

The platform team of a fictional cloud storage company that buys hundreds of thousands of drives.

Consumer	Question	Output
Procurement	Which models fail least per drive-year? How does reliability change with age?	AFR marts, survival curves
Data center operations	How healthy is the fleet now?	Daily fleet health with rolling 30-day AFR
Reliability / ML	Can SMART data predict failure 30 days out?	Point-in-time feature and label table

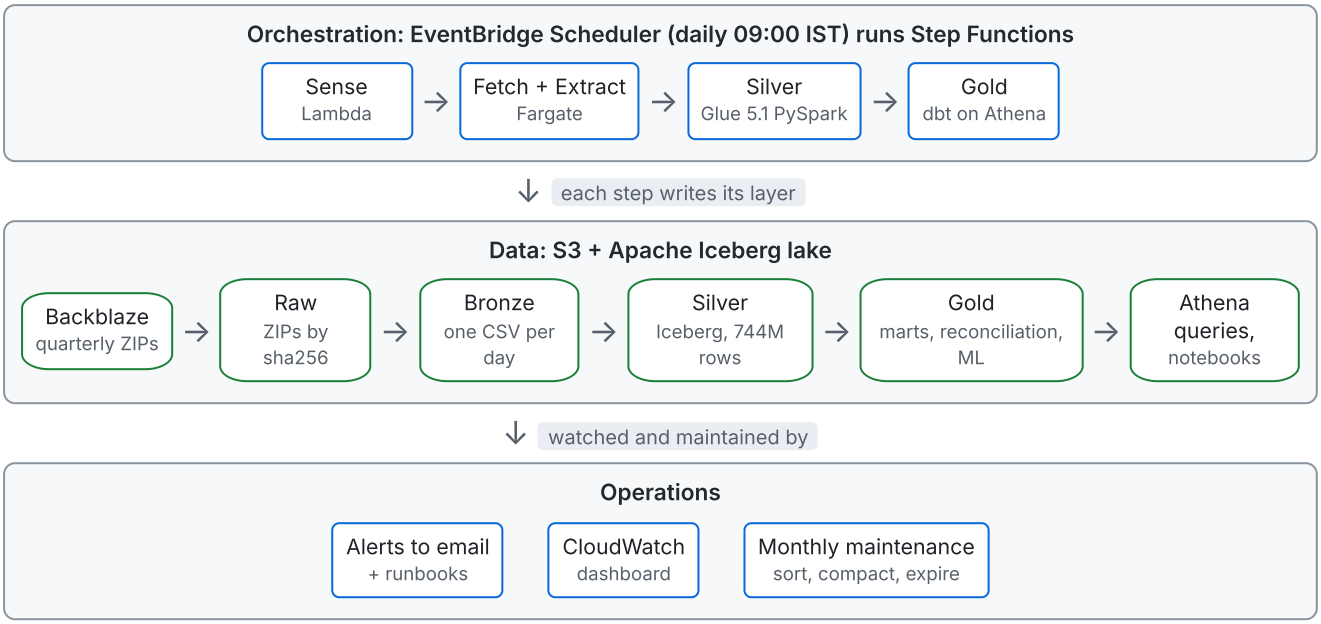
4. Requirements and results

ID	Requirement	Result
FR1	Detect new quarterly ZIPs automatically	✅ Daily sensor Lambda
FR2	Archive every source file byte for byte	✅ Content-addressed raw store (sha256)
FR3	One bronze file per day as the unit of work	✅
FR4	Parse by header name, never by position	✅ 12 layouts → one silver table
FR5	Drift policy	✅ In the contract; proven by an automated drill (§9)
FR6	Quarantine bad rows; record empty days	✅ 8 quarantined rows in all history; empty days recorded as warnings
FR7	Gold: dimensions, AFR, fleet health	✅ (SCD2 drive location deferred, issue #76)
FR8	ML tables	✅ `surv_drive`, `fct_drive_failure`, `feat_drive_day`
FR9	Reconcile AFR with published figures	✅ All 78 figures
FR10	Backfill or replay any range	✅ Backfill state machine; pipeline quarter override

ID	Requirement	Result
FR11	Run metadata and DQ as tables	✅ `ops.bronze_days`, `ops.silver_days`, `silver.quarantine`

NFR	Target	Measured
Backfill	< 3 h wall, < ~$20	≈54 DPU-h, ≈10.8 h of Glue time run one quarter at a time (wall-time target missed on purpose: concurrency 1 protects Free plan credits)
Incremental	New quarter in gold within 24 h	✅ Daily schedule; a quarter takes 8–31 min in silver plus the dbt build
Idempotency	Rerun → identical results	✅ Gold fingerprint identical after a full Q2 2019 rerun
Correctness	AFR within tolerance	✅ 74/78; 4 documented gaps
Query	One quarter's AFR < 10 s and < 2 GB	✅ ~1.2 s, 20 MB; one-drive history ~2 s, 430 MB
Operability	Every alarm has a runbook	✅

5. Architecture



Step-by-step detail:

```
EventBridge Scheduler  dsl-dev-pipeline-daily   cron(0 9 * * ? *)  Asia/Kolkata
        │
        ▼
Step Functions  dsl-dev-pipeline
    Sense      Lambda dsl-dev-sensor: is the next quarterly ZIP published?
    NewQuarter Choice: no → NothingNew (recorded no-op)
    Fetch      Fargate (image dsl-dev-fetcher): download ZIP, store content-addressed
    Extract    Fargate: one gzip CSV per day into bronze; header checked against the
contract
    Bookkeeping Fargate: ops.bronze_days
    Silver     Glue 5.1 PySpark (.sync), retry on ConcurrentRunsExceeded
    Gold       Fargate, same image: dbt build on Athena (models, seeds, tests)
    Done

Step Functions  dsl-dev-backfill
    Map over {"quarters":[{quarter,from,to}...]}, MaxConcurrency 1, Glue directly,
    Catch → Failed (other quarters continue), same retry

EventBridge Scheduler  dsl-dev-silver-maintenance-monthly  cron(0 2 1 * ? *)  Asia/Kolkata
    Glue dsl-dev-silver-maintenance: sort small files, compact, expire snapshots
```

Data: Backblaze ZIP → **raw** (immutable) → **bronze** (one CSV per day) → **silver** (Iceberg) → **gold** (Iceberg, built by dbt) → Athena and notebooks.

Design principles

1. **The day is the unit of work.** The backfill and the quarterly load share one code path; every write replaces exactly the days it processes.
2. **Raw is immutable** and content-addressed; it is the replay source.
3. **Rebuild beats patch** for gold: a full `dbt build` every run.
4. **Drift is governed by a versioned contract**, not by code changes.
5. **Measure before optimising**; keep the numbers.

Technology choices

Concern	Choice	Notes
Fetch / extract	ECS Fargate	GB-scale ZIPs, no 15-minute limit; public subnet with a public IP, no NAT Gateway
Transform	AWS Glue 5.1 PySpark	G.1X × 5 workers for loads, × 10 for maintenance
Table format	Apache Iceberg (Glue catalog)	Branches for write-audit-publish, atomic commits, partition evolution, time travel
Gold modelling	dbt-athena (dbt-core 1.12)	Runs inside the fetcher image (`fetcher/gold.py`)
Orchestration	Step Functions + EventBridge Scheduler	No always-on scheduler to pay for
Query	Athena	Separate workgroup `dsl-dev-dbt` with a 50 GB per-query cap
IaC / CI	Terraform + GitHub Actions	Bootstrap (IAM) applied locally; dev applied by CI
Language / tooling	Python, uv, ruff, pytest	

6. Storage layers

Layer	Location	Format	Layout	Mutability
Raw	`s3://dsl-dev-raw-…`	ZIP as published	By file and sha256	Immutable
Bronze	`s3://dsl-dev-bronze-…`	gzip CSV, one per day	`date=` / `src=`	Immutable; never registered for analysts
Silver	`s3://dsl-dev-lake-…/silver`	Iceberg, Parquet	Year partitions 2013–2024, **month partitions from 2025** (partition evolution), **sorted by serial_number**, ~200 MB files	Replace-days only
Gold	`s3://dsl-dev-lake-…/gold`, schema `dsl_dev_gold`	Iceberg (dbt)	Mostly unpartitioned; `feat_drive_day` by month	Full rebuild
Ops	`dsl_dev_ops`, plus `ops/bronze_days/bronze_days.jsonl`	Iceberg + JSONL	—	Append / replace per day

dbt uses `s3_data_naming: schema_table_unique`; it is needed for its high-availability rebuild (build `__ha`, swap, drop the backup).

7. Contract and ops tables

Contract `contracts/drive_stats_daily.v1.yaml` (version 1.3.0) defines:

- **Columns**: name, type, `required`, `since`.
- **SMART pattern** `^smart_(\d+)_(normalized|raw)$`.
- **Row rules**: core columns quarantine the row when broken; detail columns are blanked or kept with a warning. Date formats are listed explicitly.
- **Day limits**: quarantined share 0.5%; 50 conflicting drives.
- **Row-count baseline**: median of the previous 7 days; below 95% → partial; beyond ±3% → warn.
- **Known source defects** (added in 1.3.0).

- **Schema drift policy**:

Change	Action
New column matching the SMART pattern	auto_add
New column not matching it	block_day
Required column missing	block_day
Optional column missing that existed earlier	warn (judged by the day's date, never by load order)
Columns reordered	info (columns are read by name)

Ops tables

Table	Contents
`ops.bronze_days`	One row per extracted day: header fingerprint, row count, status (blocked days never reach silver)
`ops.silver_days`	One verdict per day: status (ok / warn / partial / blocked), blocks, warnings, bronze_rows, silver_rows, quarantined_rows, run_id
`silver.quarantine`	Rejected rows with a reason

Silver totals:

- 4,830 days: 3,319 ok, 1,429 warn, 82 partial, 0 blocked.
- 4,826 days have rows. The other four are 3 empty source files plus the 2022-12-10 defect.
- The fingerprint timeline check passed: no column has values before its layout starts.

8. Gold layer

dbt project `dbt/`, profile `backblaze`, schema `dsl_dev_gold`, workgroup `dsl-dev-dbt`.

Model	Grain	Purpose
`fleet_daily`	Date	Drives and failures per day
`dim_drive_model`	Model	Manufacturer (seed pattern match), usual capacity
`fct_model_daily`	Date × model	Drive-days and failures
`fct_drive_quarter`	Serial × quarter	Capacity, last day, at-quarter-end flag, boot-drive flag
`mart_afr_quarterly`	Model × quarter	AFR, unfiltered, with day statuses
`mart_afr_quarterly_fleet`	Quarter	Fleet AFR with era inclusion rules
`mart_afr_yearly_fleet`	Year	Yearly fleet AFR
`mart_fleet_health_daily`	Date	Rolling 30-day AFR
`rec_published_afr`	Published figure	Ours vs published, difference, tolerance, status
`surv_drive`	Drive	Survival analysis input (§10)
`fct_drive_failure`	Failed drive	SMART state at failure (§10)
`feat_drive_day`	Drive × day	ML features and label (§10)

Seeds:

- `model_manufacturers.csv`: pattern → manufacturer.
- `published_afr.csv`: 78 published figures, each with `status` check or known_gap and a source URL.

AFR = failures / (drive-days / 365) × 100. A failed drive's last row has `failure = 1`.

Reconciliation

Backblaze's inclusion rules had to be reverse-engineered, and they changed by era:

- **Boot drives** (`capacity_bytes` < 1 TB, decided per drive per quarter) are excluded.
- **Small-population models** are excluded below a minimum:

Era	Minimum
Before 2019	45 drives at quarter end
2019–2022	60 drives at quarter end
2023 onwards	more than 100 drives **and** more than 10,000 drive-days

- **Tolerance:** 0.01 for model figures, 0.05 for fleet figures. Backblaze's own posts disagree with each other by up to 0.06.

Result: 78 figures (32 quarterly fleet, 11 yearly fleet, 35 model); 74 within tolerance. The 4 known gaps:

Figure	Published	Ours	Note
2015 yearly fleet	2.47	~2.78	Unexplained
2014 yearly fleet	6.39	6.28 (6.38 with no minimum)	Era rule for 2014
ST8000DM002, 2016-Q4	—	—	Published figure likely covers all 8 TB Seagate models together
ST10000NM0086, 2026-Q2	22 failures	23 failures	The public files contain 23 distinct failed drives

Wrong turns (kept in `docs/reconciliation.md`):

- "No slot means boot drive" dropped the failure-day rows of real data drives.
- "Never in a slot all quarter" caught data drives that later failed.

9. Data quality

Layer	Check	Action
Extract	Header vs contract drift policy; filename date parses	Block the day (file still stored)
Silver row	Casts; `failure` ∈ {0,1}; serial not null	Quarantine the row
Silver day	Quarantined share > 0.5% or > 50 conflicting drives	Block
Silver day	Row count vs 7-day median	Partial or warn, never block
Silver	Bronze rows = silver + quarantined + deduplicated	Balance check (known-defect days excluded)
Silver publish	Write-audit-publish on an Iceberg branch; only audited, unblocked days are published	—
Gold	dbt tests on every model, including totals matching silver, one row per key, no new unknown models, reconciliation within tolerance	Build fails

Known source defect: 2022-12-10.

- It is a copy of 2022-12-11: its date column says 12-11, and 235,615 of 235,623 rows are identical.
- Median power-on hours jump 47 hours from 12-09, which is two days, not one.
- Contract 1.3.0 lists it, so silver skips it with a warning, records the real `bronze_rows`, and sets `silver_rows = 0`.

Drift drill. `tests/test_drift_drill.py` runs the full policy against the real contract and functions:

1. A header with `rack_temperature` is blocked by `check_header`, which names the column.
2. `select_days` loads nothing.
3. The contract (a copy) is updated to allow the column.
4. The same header passes and the day is selected.

A new `smart_999_*` pair passes with no change. The drill runs in CI on every pull request. Walkthrough: `docs/drift-drill.md`.

10. ML-ready tables

Table	Grain	Contents
`surv_drive`	Drive	`duration_days`, `event` (1 failed, 0 censored), `end_reason` (failed, still_running, removed)
`fct_drive_failure`	Failed serial (first failure)	SMART 5/187/188/197/198 on the failure day, their change vs 7–14 and 30–37 days earlier, age from power-on hours, `any_warning`
`feat_drive_day`	Hard drive × day, 2025-07-01 → 2026-06-30	The same SMART values, 7- and 30-day changes and 30-day maximum, age, and `label_fail_30d`

Rules

- **Death = first failure.** 3.6% of failed serials reappear. Using power-on hours as an odometer, 93% are the same physical drive and 7% are a different drive reusing the serial.
- `surv_drive` excludes drives already running on 2013-04-10, because their start date is unknown.
- **Point-in-time features:** window functions look only backwards, and change values need a reading within a bounded window (7–14 or 30–37 days earlier). Otherwise they are null rather than misleading.

- **Labels:**

  - `label_fail_30d` = 1 if the first failure falls in d+1..d+30;
  - 0 if not, and all 30 days are observed;
  - **null if the window runs past the data end**.

  Rows on or after the failure day are excluded. - `feat_drive_day` excludes boot drives (under 1 TB), whose SMART semantics differ.

Results

- `surv_drive`: 355,034 still running, 105,824 removed, 30,547 failed.
- `fct_drive_failure`: 33,565 rows, equal to the distinct failed serials in silver (tested). 59–78% of failed hard drives showed a SMART warning on their last day.
- `feat_drive_day`: 121,849,508 rows; labels 1 / 0 / null = 118,155 / 111,220,051 / 10,511,302 (~0.1% positive).
- Kaplan–Meier by manufacturer (`analysis/km_by_manufacturer.py`, `docs/evidence/km-by-manufacturer.png`): HGST ~90% surviving at 9 years; Toshiba wears out after ~6 years; Seagate declines steadily.

Leakage tests (dbt):

- no 0 label within 30 days of the data end;
- no rows on or after failure;

- labels agree with `fct_drive_failure`;
- for a sample week, features recomputed directly from silver using only past rows must match.

11. Operations

Alerts (SNS topic `dsl-dev-alerts`, email):

Alert	Trigger
`dsl-dev-pipeline-failed`	Step Functions executions failed > 0 in an hour
`dsl-dev-pipeline-not-running`	No pipeline start for 2 days
`dsl-dev-sensor-errors`	Sensor Lambda errors > 0 in an hour
`dsl-dev-glue-job-failed`	EventBridge rule: any `dsl-dev-*` Glue run FAILED, TIMEOUT or ERROR (also inside the backfill, which catches failures)

- Each alert links to a runbook in `docs/runbooks/`.
- The email subscription is created by hand so the address stays out of the public repository.

Dashboard `dsl-dev-pipeline-health` shows:

- alarm status;
- pipeline runs per day and duration;
- backfill runs;
- sensor calls and errors.

Monthly maintenance (`glue/silver_maintenance_entry.py`):

1. Keep the serial_number write order.
2. Rewrite files under 64 MB into ~256 MB sorted files.
3. Expire snapshots older than 7 days. `retain_last` is 1, because the sort committed in chunks and any older snapshot would keep pre-sort files alive.
4. List orphan files (dry run).

The `--rewrite_all` and `--evolve_months` switches exist for one-off runs.

12. Retry and idempotency

Stage	Idempotency	Retry
Fetch	Content-addressed keys: same bytes, same key	Task retry
Extract	Keyed by date and source hash	Rerun
Silver	Replaces exactly the listed days: delete + append on a write-audit-publish branch (ADR 0002)	`Glue.ConcurrentRunsExceededException`: 60 s, backoff 1.5, 10 attempts
Gold	Full `dbt build`	Rerun

Data-validation failures are never retried. Proof of idempotency: the gold fingerprint before and after a full Q2 2019 rerun was identical (729,680,649 · 31,669 · 3,394 · 744,290,059 · 74).

13. Performance (measured)

Change	Before	After
Silver load, one quarter: `.cache()` on reused DataFrames (Spark laziness ran the dedupe 6× and parsed bronze 3×)	1,944 s, 1.08 DPU-h	502 s, 0.70 DPU-h
Silver files: sort by serial_number and compact	343 files, avg 71 MB, ~24.3 GB	91 files, avg 188 MB, ~17.1 GB

Bytes scanned per query, as recorded by `analysis/benchmark_silver.py`. Run time is noisy, so MB scanned is the main metric.

Query	Unsorted	Sorted, year partitions	Sorted, months for 2025+ (current)
Q1 AFR by model, one quarter	3.2 MB	62.9 MB	**20.3 MB**
Q2 one drive's full history	4,043.5 MB	115.1 MB	**429.9 MB**
Q3 one model across 13 years	46.1 MB	180.7 MB	**212.0 MB**

Lessons

- **Sorting enables file skipping** through Parquet min/max statistics, and it **improves compression**: each drive's days sit together.
- **A table has one physical order.** Before the sort, files happened to be grouped by date, so date queries were cheap; the sort broke that.
- **Partitioning and sorting compete.** Once a month fits in one or two files, each file spans all serials and cannot be skipped. Month partitions were kept because quarterly reads are the everyday workload.
- **Measure the maintenance too.** The first scheduled-mode run spent 21.9 of 23 minutes rewriting files that were already fine, because Iceberg's default "too small" threshold is ~192 MB. A 64 MB minimum fixed it.

Backfill:

- 53 quarters, ~38,900 s of Glue time, ~54 DPU-h.
- Per-quarter time grows with the fleet (154 s in 2013 → 1,869 s in 2026-Q2).
- The first attempt failed 50 quarters within a second on Glue concurrency, until a retry was added.

14. Security and IAM

Each component has its own role with least privilege:

Role	Used by
`dsl-dev-sensor`	Sensor Lambda
`dsl-dev-pipeline`	Step Functions
`dsl-dev-scheduler`	Daily pipeline schedule; monthly maintenance
`dsl-dev-fetcher-task`	Fargate task, including Athena / Glue / S3 for dbt (workgroup `dsl-dev-dbt` only)
`dsl-dev-silver-glue`	Glue silver and maintenance jobs
CI user `github_apply`	Policies scoped to `dsl-dev-*` resources

Other points:

- CI uses stored keys (ADR 0001); OIDC is the planned upgrade.
- No NAT Gateway.
- The data contains no personal information (drive serials are not PII).

15. CI/CD and testing

Workflow	Trigger	Does
`ci.yml`	Pull request	ruff, pytest, Terraform fmt / validate / plan
`terraform-dev.yml`	Merge (and after an image build)	Applies dev Terraform
`deploy-silver.yml`	Changes to `silver/`, `glue/`, `contracts/`, the deploy script	Uploads the Glue code and contract
`fetcher-image.yml`	Changes to `fetcher/`, `dbt/`, `contracts/`, dependencies	Builds and pushes `dsl-dev-fetcher:<sha7>` (immutable tags); the task uses the newest digest

Testing level	What
Unit (pytest)	Header parsing, drift policy, casting, dedupe, audits, sensor, extract, publish, drift drill
dbt	Data tests on every gold model; reconciliation; leakage tests
Idempotency	Full rerun fingerprint
Performance	Benchmark script with saved before / after results

Every change goes through branch → pull request → checks → merge.

16. Repository layout

Folder	Contents
`contracts/`	Source data contract
`fetcher/`	Sensor, fetch / extract / bookkeeping, gold (dbt) runner, Dockerfile
`silver/`	Glue PySpark package: read, cast, dedupe, audit, write, publish
`glue/`	Glue entry scripts: silver load, maintenance
`dbt/`	Models (fleet, dims, facts, marts, reconciliation, ml), seeds, tests
`infra/`	Terraform: `bootstrap/` (IAM, applied locally), `envs/dev/` (applied by CI)
`analysis/`	Kaplan–Meier notebook, silver benchmark
`docs/`	ADRs, runbooks, evidence, performance, profiling, pipeline, reconciliation, ML tables, drift drill
`tests/`	pytest

17. Deviations from the original plan

Plan	Built	Why
OIDC for CI	Stored keys (ADR 0001)	Faster start; OIDC deferred

Plan	Built	Why
Weekly sensor that re-checks old ZIPs	Daily sensor for the next quarter only	Republication is handled by content addressing on re-fetch
Overwrite-by-filter	Replace-days: delete + append on a WAP branch (ADR 0002)	Works with Iceberg branches and is idempotent
Row count ±15% blocks	Median-based partial / warn, never block	Source anomalies are Backblaze's, not ours: label them, don't stop the pipeline
`pipeline_runs` / `dq_results` / `observed_schemas`	`silver_days`, `bronze_days`, header layouts	One verdict per day was enough
Separate dbt image	dbt inside the fetcher image	One image, one build
Reconcile sampled quarters	All 78 published figures	Sampling hid the era-specific rules
Feature job on Glue	`feat_drive_day` in dbt on Athena, one-year scope	Fits at this scope; reviewable SQL and tests
Silver by month, sorted by (model, serial)	Years, then months from 2025; sorted by serial only	Benchmarks: only serial lookups were costly; month folders for old years would be ~20 MB
SCD2 drive location	Deferred (issue #76)	About 0.5% of drives ever move
Prod environment	Deferred (issue #81)	Cost; dev proves the full system

18. Known gaps and future work

- Prod environment with artifact promotion and approval (#81).
- SCD2 `dim_drive_location` (#76). Rules already decided: ignore empty-slot days; `valid_to` = the day before the next stay.
- The 2015 yearly AFR gap is unexplained.
- The sensor handles quarterly ZIPs only (2013–2015 were loaded by backfill).
- Glue time per quarter grows with the fleet.
- Full-history `feat_drive_day` would need an incremental (month-by-month) build.
- Move CI to OIDC.
- Pin the `uv` base image in the Dockerfile.

After 1 Nov 2026: confirm the first scheduled maintenance run deleted the pre-sort files (expire step output). When Q3 2026 arrives, confirm the daily run is green end to end.

19. Interview stories (with numbers)

1. **Spark laziness:** 1,944 s → 502 s by caching reused DataFrames.
2. **The backfill concurrency failure:** 50 quarters failed in one second; fixed with a targeted retry. Data errors are never retried.
3. **A duplicated source file** (2022-12-10), proven with power-on hours, handled by a contract mechanism instead of a code hack.
4. **Reconciling all 78 published AFR figures**: reverse-engineering era-specific inclusion rules, including the wrong turns.
5. **Idempotency proven** by fingerprints before and after a full rerun.
6. **Testing assumptions:** "failed drives never come back" was false (3.6%); power-on hours separated real returns from reused serials.
7. **Point-in-time labels:** null, not 0, at the horizon; leakage tests recompute features from source.

8. **Layout trade-offs measured:** sort by serial (35× less data for lookups, 30% smaller), then month partitions (3× cheaper quarterly reads, 3.7× costlier lookups). Partitioning and sorting compete.
9. **Drift policy drill:** a new column is blocked, the contract updated, the day replayed, all as an automated test in CI.
10. **Measuring maintenance:** the slow step was the rewrite, not the cleanup, as found from log timestamps; fixed with one threshold.